

✓ Análisis estadístico OE9 — TerraRover-Gen

Este notebook ejecuta el análisis estadístico completo del OE9 (Robustez física).

Instrucciones de uso en Google Colab:

- Sube los 12 archivos CSV al área de archivos del notebook (icono de carpeta a la izquierda):
 - 6 del agente RL: baseline V3 (`HuskyAgent2_metricas_V3_F3_02.csv`), baseline BumpyGround (`HuskyAgent2_metricas_BumpyGround.csv`), Sticky V3 (`HuskyAgent2_metricas_TerrainSticky_V3_F3_02.csv`), Sticky BumpyGround (`HuskyAgent2_metricas_TerrainSticky_BumpyGround.csv`), FL2000 V3 (`HuskyAgent2_metricas_FL2000_V3_F3_02.csv`), FL2000 BumpyGround (`HuskyAgent2_metricas_FL2000_BumpyGround.csv`).
 - 6 del heurístico: análogos con prefijo `HuskyHeuristic_metricas_`.
- Ejecuta las celdas de arriba abajo (Shift+Enter).
- Los resultados se imprimen en la salida y se guardan en `resultados_OE9.json`.

✓ 1. Dependencias

```
!pip install statsmodels --quiet
```

✓ 2. Imports y configuración

```
import os
import json
import numpy as np
import pandas as pd
from scipy import stats
from statsmodels.stats.contingency_tables import mcnemar
from statsmodels.stats.proportion import proportion_confint

CSV_DIR = '.'

FILES = {
    ('RL', 'V3', 'baseline'): 'HuskyAgent2_metricas_V3_F3_02.csv',
    ('RL', 'V3', 'sticky'): 'HuskyAgent2_metricas_TerrainSticky_V3_F3_02.csv',
    ('RL', 'V3', 'fl2000'): 'HuskyAgent2_metricas_FL2000_V3_F3_02.csv',
    ('RL', 'BumpyGround', 'baseline'): 'HuskyAgent2_metricas_BumpyGround.csv',
    ('RL', 'BumpyGround', 'sticky'): 'HuskyAgent2_metricas_TerrainSticky_BumpyGround.csv',
    ('RL', 'BumpyGround', 'fl2000'): 'HuskyAgent2_metricas_FL2000_BumpyGround.csv',
    ('HEU', 'V3', 'baseline'): 'HuskyHeuristic_metricas_V3_F3_02.csv',
    ('HEU', 'V3', 'sticky'): 'HuskyHeuristic_metricas_TerrainSticky_V3_F3_02.csv',
    ('HEU', 'V3', 'fl2000'): 'HuskyHeuristic_metricas_FL2000_V3_F3_02.csv',
    ('HEU', 'BumpyGround', 'baseline'): 'HuskyHeuristic_metricas_BumpyGround.csv',
    ('HEU', 'BumpyGround', 'sticky'): 'HuskyHeuristic_metricas_TerrainSticky_BumpyGround.csv',
    ('HEU', 'BumpyGround', 'fl2000'): 'HuskyHeuristic_metricas_FL2000_BumpyGround.csv',
}

CONFIG_LABEL = {
    'baseline': 'μ=0.3, FL=1000 (baseline)',
    'sticky': 'μ=1.5, FL=1000 (fricción alta)',
    'fl2000': 'μ=0.3, FL=2000 (par alto)',
}
```

✓ 3. Funciones de análisis

```
def load(system, terrain, config):
    fname = FILES[(system, terrain, config)]
    df = pd.read_csv(os.path.join(CSV_DIR, fname), sep=';')
    return df.set_index('episodio').sort_index()

def align(df_a, df_b):
    common = df_a.index.intersection(df_b.index)
    return df_a.loc[common], df_b.loc[common]

def wilson_ci(successes, n, alpha=0.05):
    if n == 0: return 0.0, 0.0
    lo, hi = proportion_confint(successes, n, alpha=alpha, method='wilson')
```

```

        return lo*100, hi*100

def mcnemar_paired(succ_a, succ_b):
    a = int(((succ_a)&(succ_b)).sum())
    b = int(((~succ_a)&(succ_b)).sum())
    c = int((succ_a&(~succ_b)).sum())
    d = int(((~succ_a)&(succ_b)).sum())
    if b+c == 0: return a, b, c, d, 1.0
    result = mcnemar([[a,b],[c,d]], exact=True)
    return a, b, c, d, float(result.pvalue)

def diff_props_paired_ci(b, c, n):
    if n == 0: return 0.0, 0.0, 0.0
    diff = (c-b)/n*100
    if b+c == 0: return diff, diff, diff
    se = np.sqrt((b+c-(c-b)**2/n)/n)/n
    return diff, ((c-b)/n - 1.96*se)*100, ((c-b)/n + 1.96*se)*100

def wilcoxon_paired(a, b):
    a, b = np.asarray(a), np.asarray(b)
    if (a-b == 0).all(): return None
    try: _, p = stats.wilcoxon(a, b, zero_method='wilcox', alternative='two-sided')
    except ValueError: return None
    n = len(a)
    z = stats.norm.ppf(1-p/2)
    return {'p': float(p), 'r': float(abs(z)/np.sqrt(n)),
            'median_a': float(np.median(a)), 'median_b': float(np.median(b)), 'n': int(n)}

def print_header(title, char='-'):
    bar = char*90
    print(f'\n{bar}\n{title}\n{bar}')

```

✓ 4. Bloque A — Intra-sistema (baseline vs perturbado, mismo agente)

Para cada (sistema, terreno, perturbación) se compara el rendimiento del MISMO sistema en baseline vs perturbado bajo seeds compartidas.

```

results = {'A_intra': {}, 'B_inter': {}, 'C_paired_clean': {}}

def analyze_intra(system, terrain, pert):
    label = CONFIG_LABEL[pert]
    print_header(f'{system} en {terrain}: baseline vs {label}', char='-')
    base = load(system, terrain, 'baseline')
    p = load(system, terrain, pert)
    base, p = align(base, p)
    n = len(base)
    sb = (base['resultado']=='SUCCESS').values
    sp = (p['resultado']=='SUCCESS').values
    nb, np_ = int(sb.sum()), int(sp.sum())
    lo_b, hi_b = wilson_ci(nb, n)
    lo_p, hi_p = wilson_ci(np_, n)
    print(f'Baseline: {nb/n*100:5.1f}% IC95% [{lo_b:5.1f}, {hi_b:5.1f}] ({nb}/{n})')
    print(f'Perturbado: {np_/n*100:5.1f}% IC95% [{lo_p:5.1f}, {hi_p:5.1f}] ({np_}/{n})')
    print(f'\nDistribución (Baseline → Perturbado):')
    for r in ['SUCCESS', 'STUCK', 'COLLISION', 'FALL', 'SPIN']:
        cb, cp = int((base['resultado']==r).sum()), int((p['resultado']==r).sum())
        print(f' {r:12s}: {cb:3d} → {cp:3d} ({cp-cb:+d})')
    a, b, c, d, pv = mcnemar_paired(sb, sp)
    diff = (np_-nb)/n*100
    _, dlo, dhi = diff_props_paired_ci(c, b, n)
    print(f'\nMcNemar pareado: p={pv:.4f} (a={a}, b={b}, c={c}, d={d})')
    print(f'Δ pert-base: {diff:+.1f}pp IC95% [{dlo:+.1f}, {dhi:+.1f}]')
    both = sb & sp
    nboth = int(both.sum())
    print(f'\nWilcoxon (n={nboth}):')
    if nboth >= 5:
        for var in ['pasos','tiempo_s','distancia_final_m','energia_total']:
            res = wilcoxon_paired(base.loc[both,var], p.loc[both,var])
            if res is None:
                print(f' {var:24s}: no calculable')
            else:
                d_str = 'base<pert' if res['median_a']<res['median_b'] else 'base>pert'
                print(f' {var:24s}: base={res["median_a"]:.82f} pert={res["median_b"]:.82f} ({d_str}) p={res["p"]:.4f} r={r}')
    return {'n':n, 'baseline_rate':nb/n*100, 'perturbed_rate':np_/n*100,
            'mcnemar_p':pv, 'diff_pp':diff, 'diff_ci':(dlo,dhi),
            'results_baseline':dict(base['resultado'].value_counts()),
            'results_perturbed':dict(p['resultado'].value_counts())}

for system in ['RL', 'HEU']:
    for terrain in ['V3', 'BumpyGround']:

```

```
for pert in ['sticky', 'fl2000']:
    results['A_intra'][f'{system}_{terrain}_{pert}'] = analyze_intra(system, terrain, pert)
```

Baseline: 89.0% IC95% [81.4, 93.7] (89/100)
 Perturbado: 89.0% IC95% [81.4, 93.7] (89/100)

Distribución (Baseline → Perturbado):

SUCCESS : 89 → 89 (+0)
 STUCK : 10 → 9 (-1)
 COLLISION : 1 → 2 (+1)
 FALL : 0 → 0 (+0)
 SPIN : 0 → 0 (+0)

McNemar pareado: p=1.0000 (a=87, b=2, c=2, d=9)

Δ pert-base: +0.0pp IC95%: [-0.4, +0.4]

Wilcoxon (n=87):

pasos : base= 841.00 pert= 904.00 (base<pert) p=0.5898 r=0.058
 tiempo_s : base= 8.41 pert= 9.04 (base<pert) p=0.5808 r=0.059
 distancia_final_m : base= 1.99 pert= 1.99 (base>pert) p=0.0946 r=0.179
 energia_total : base= 2334.00 pert= 2373.00 (base<pert) p=0.6174 r=0.054

 HEU en BumpyGround: baseline vs $\mu=1.5$, FL=1000 (fricción alta)

Baseline: 70.0% IC95% [60.4, 78.1] (70/100)
 Perturbado: 73.0% IC95% [63.6, 80.7] (73/100)

Distribución (Baseline → Perturbado):

SUCCESS : 70 → 73 (+3)
 STUCK : 10 → 4 (-6)
 COLLISION : 5 → 2 (-3)
 FALL : 15 → 21 (+6)
 SPIN : 0 → 0 (+0)

McNemar pareado: p=0.6476 (a=62, b=11, c=8, d=19)

Δ pert-base: +3.0pp IC95%: [+2.1, +3.9]

Wilcoxon (n=62):

pasos : base= 904.50 pert= 877.50 (base>pert) p=0.0005 r=0.444
 tiempo_s : base= 9.04 pert= 8.78 (base>pert) p=0.0005 r=0.443
 distancia_final_m : base= 1.99 pert= 1.99 (base>pert) p=0.1609 r=0.178
 energia_total : base= 2548.50 pert= 2556.85 (base<pert) p=0.0002 r=0.479

 HEU en BumpyGround: baseline vs $\mu=0.3$, FL=2000 (par alto)

Baseline: 70.0% IC95% [60.4, 78.1] (70/100)
 Perturbado: 74.0% IC95% [64.6, 81.6] (74/100)

Distribución (Baseline → Perturbado):

SUCCESS : 70 → 74 (+4)
 STUCK : 10 → 6 (-4)
 COLLISION : 5 → 7 (+2)
 FALL : 15 → 13 (-2)
 SPIN : 0 → 0 (+0)

McNemar pareado: p=0.3438 (a=67, b=7, c=3, d=23)

Δ pert-base: +4.0pp IC95%: [+3.4, +4.6]

Wilcoxon (n=67):

5. Bloque B — Inter-sistema (RL vs HEU bajo cada configuración)

```
def analyze_inter(terrain, config):
    label = CONFIG_LABEL[config]
    print_header(f'RL vs HEU en {terrain} con {label}', char='-')
    rl = load('RL', terrain, config)
    heu = load('HEU', terrain, config)
    rl, heu = align(rl, heu)
    n = len(rl)
    sr = (rl['resultado']=='SUCCESS').values
    sh = (heu['resultado']=='SUCCESS').values
    nr, nh = int(sr.sum()), int(sh.sum())
    lo_r, hi_r = wilson_ci(nr, n)
    lo_h, hi_h = wilson_ci(nh, n)
    print(f'RL: {nr/n*100:5.1f}% IC95% [{lo_r:5.1f}, {hi_r:5.1f}] ({nr}/{n})')
    print(f'HEU: {nh/n*100:5.1f}% IC95% [{lo_h:5.1f}, {hi_h:5.1f}] ({nh}/{n})')
    a, b, c, d, pv = mcnemar_paired(sr, sh)
    diff = (nr-nh)/n*100
    _, dlo, dhi = diff_props_paired_ci(b, c, n)
    print(f'\nMcNemar pareado RL vs HEU: p={pv:.4f}')
    print(f'Tabla 2x2: ambos+={a}, HEU+/RL-={b}, RL+/HEU-={c}, ambos-={d}')
    print(f'Δ RL-HEU: {diff:+.1f}pp IC95%: [{dlo:+.1f}, {dhi:+.1f}]')
    # Wilcoxon pareado en SUCCESS común
```

```

both = sr & sh
nboth = int(both.sum())
print(f'\nWilcoxon pareado en SUCCESS común (n={nboth}):')
wilcox = {}
if nboth >= 5:
    for var in ['pasos', 'tiempo_s', 'distancia_final_m', 'energia_total']:
        res = wilcoxon_paired(rl.loc[both, var], heu.loc[both, var])
        if res is None:
            print(f' {var:24s}: no calculable')
            wilcox[var] = None
        else:
            d_str = 'RL<HEU' if res['median_a'] < res['median_b'] else 'RL>HEU'
            print(f' {var:24s}: RL={res["median_a"]:.2f} HEU={res["median_b"]:.2f} ({d_str}) p={res["p"]:.4f} r={res["r"]:.4f}')
            wilcox[var] = res
    else:
        print(f' (insuficientes pares con SUCCESS común)')
return {'n':n, 'rl_rate':nr/n*100, 'heu_rate':nh/n*100,
        'mcnemar_p':pv, 'diff_pp':diff, 'diff_ci':(dlo, dhi),
        'n_both_success':nboth, 'wilcoxon':wilcox}

for terrain in ['V3', 'BumpyGround']:
    for config in ['baseline', 'sticky', 'fl2000']:
        results['B_inter'][f'{terrain}_{config}'] = analyze_inter(terrain, config)

```

RL vs HEU en V3 con $\mu=0.3$, FL=1000 (baseline)

RL: 87.0% IC95% [79.0, 92.2] (87/100)
HEU: 89.0% IC95% [81.4, 93.7] (89/100)

McNemar pareado RL vs HEU: p=0.8145
Tabla 2x2: ambos+=79, HEU+/RL-=10, RL+/HEU-=8, ambos-=3
 Δ RL-HEU: -2.0pp IC95%: [-2.8, -1.2]

Wilcoxon pareado en SUCCESS común (n=79):

pasos	: RL= 1580.00 HEU= 782.00 (RL>HEU) p=0.0000 r=0.809
tiempo_s	: RL= 15.80 HEU= 7.82 (RL>HEU) p=0.0000 r=0.809
distancia_final_m	: RL= 2.00 HEU= 1.99 (RL>HEU) p=0.2132 r=0.140
energia_total	: RL= 2378.23 HEU= 2223.00 (RL>HEU) p=0.0785 r=0.198

RL vs HEU en V3 con $\mu=1.5$, FL=1000 (fricción alta)

RL: 98.0% IC95% [93.0, 99.4] (98/100)
HEU: 92.0% IC95% [85.0, 95.9] (92/100)

McNemar pareado RL vs HEU: p=0.0703
Tabla 2x2: ambos+=91, HEU+/RL-=1, RL+/HEU-=7, ambos-=1
 Δ RL-HEU: +6.0pp IC95%: [+5.5, +6.5]

Wilcoxon pareado en SUCCESS común (n=91):

pasos	: RL= 960.00 HEU= 747.00 (RL>HEU) p=0.0000 r=0.627
tiempo_s	: RL= 9.60 HEU= 7.47 (RL>HEU) p=0.0000 r=0.628
distancia_final_m	: RL= 2.00 HEU= 1.99 (RL>HEU) p=0.0950 r=0.175
energia_total	: RL= 1995.23 HEU= 2073.00 (RL<HEU) p=0.1143 r=0.166

RL vs HEU en V3 con $\mu=0.3$, FL=2000 (par alto)

RL: 93.0% IC95% [86.3, 96.6] (93/100)
HEU: 89.0% IC95% [81.4, 93.7] (89/100)

McNemar pareado RL vs HEU: p=0.3438
Tabla 2x2: ambos+=86, HEU+/RL-=3, RL+/HEU-=7, ambos-=4
 Δ RL-HEU: +4.0pp IC95%: [+3.4, +4.6]

Wilcoxon pareado en SUCCESS común (n=86):

pasos	: RL= 1020.50 HEU= 897.50 (RL>HEU) p=0.0031 r=0.319
tiempo_s	: RL= 10.20 HEU= 8.97 (RL>HEU) p=0.0031 r=0.319
distancia_final_m	: RL= 1.99 HEU= 1.99 (RL>HEU) p=0.0014 r=0.345
energia_total	: RL= 2074.30 HEU= 2373.00 (RL<HEU) p=0.0001 r=0.413

RL vs HEU en BumpyGround con $\mu=0.3$, FL=1000 (baseline)

RL: 31.0% IC95% [22.8, 40.6] (31/100)
HEU: 70.0% IC95% [60.4, 78.1] (70/100)

McNemar pareado RL vs HEU: p=0.0000
Tabla 2x2: ambos+=26, HEU+/RL-=44, RL+/HEU-=5, ambos-=25
 Δ RL-HEU: -39.0pp IC95%: [-40.1, -37.9]

6. Bloque C — Análisis pareado limpio en BumpyGround

Excluye seeds donde cualquiera de los dos sistemas tuvo FALL (limpieza simétrica).

```
def analyze_clean(config):
    label = CONFIG_LABEL[config]
    print_header(f'Análisis pareado limpio: RL vs HEU en BumpyGround con {label}', char='-')
    rl = load('RL', 'BumpyGround', config)
    heu = load('HEU', 'BumpyGround', config)
    rl, heu = align(rl, heu)
    n_orig = len(rl)
    fall_rl = (rl['resultado']=='FALL').values
    fall_heu = (heu['resultado']=='FALL').values
    excl = fall_rl | fall_heu
    keep = ~excl
    n_excl = int(excl.sum())
    n = int(keep.sum())
    rl_c, heu_c = rl.iloc[keep], heu.iloc[keep]
    print(f'Excluidos: {n_excl} | Brutos: {n_orig} → Válidos: {n}\n')
    sr = (rl_c['resultado']=='SUCCESS').values
    sh = (heu_c['resultado']=='SUCCESS').values
    nr, nh = int(sr.sum()), int(sh.sum())
    lo_r, hi_r = wilson_ci(nr, n)
    lo_h, hi_h = wilson_ci(nh, n)
    print(f'RL: {nr/n*100:5.1f}% IC95% [{lo_r:5.1f}, {hi_r:5.1f}] ({nr}/{n})')
    print(f'HEU: {nh/n*100:5.1f}% IC95% [{lo_h:5.1f}, {hi_h:5.1f}] ({nh}/{n})')
    print(f'\nDistribución sobre conjunto limpio:')
    for r in ['SUCCESS', 'STUCK', 'COLLISION', 'FALL', 'SPIN']:
        cr = int((rl_c['resultado']==r).sum())
        ch = int((heu_c['resultado']==r).sum())
        print(f' {r:12s}: RL={cr:3d} HEU={ch:3d}')
    a, b, c, d, pv = mcnemar_paired(sr, sh)
    diff = (nr-nh)/n*100
    _, dlo, dhi = diff_props_paired_ci(b, c, n)
    print(f'\nMcNemar pareado RL vs HEU (limpio): p={pv:.4f}')
    print(f'Δ RL-HEU: {diff:+.1f}pp IC95% [{dlo:+.1f}, {dhi:+.1f}]')
    # Wilcoxon pareado en SUCCESS común sobre conjunto limpio
    both = sr & sh
    nboth = int(both.sum())
    print(f'\nWilcoxon pareado en SUCCESS común (n={nboth}):')
    wilcox = {}
    if nboth >= 5:
        for var in ['pasos', 'tiempo_s', 'distancia_final_m', 'energia_total']:
            res = wilcoxon_paired(rl_c.loc[both, var], heu_c.loc[both, var])
            if res is None:
                print(f' {var:24s}: no calculable')
                wilcox[var] = None
            else:
                d_str = 'RL<HEU' if res['median_a'] < res['median_b'] else 'RL>HEU'
                print(f' {var:24s}: RL={res["median_a"]:8.2f} HEU={res["median_b"]:8.2f} ({d_str}) p={res["p"]:.4f} r={re:
                wilcox[var] = res
    else:
        print(f' (insuficientes pares con SUCCESS común)')
    return {'n_orig': n_orig, 'n_excluded': n_excl, 'n': n,
            'rl_rate': nr/n*100, 'heu_rate': nh/n*100,
            'mcnemar_p': pv, 'diff_pp': diff, 'diff_ci': (dlo, dhi),
            'n_both_success': nboth, 'wilcoxon': wilcox,
            'results_rl_clean': dict(rl_c['resultado'].value_counts()),
            'results_heu_clean': dict(heu_c['resultado'].value_counts())}

for config in ['sticky', 'fl2000']:
    results['C_paired_clean'][f'BumpyGround_{config}_clean'] = analyze_clean(config)
```

Análisis pareado limpio: RL vs HEU en BumpyGround con $\mu=1.5$, FL=1000 (fricción alta)

Excluidos: 26 | Brutos: 100 → Válidos: 74

RL: 23.0% IC95% [14.9, 33.7] (17/74)
HEU: 93.2% IC95% [85.1, 97.1] (69/74)

Distribución sobre conjunto limpio:

SUCCESS	: RL= 17	HEU= 69
STUCK	: RL= 54	HEU= 4
COLLISION	: RL= 3	HEU= 1
FALL	: RL= 0	HEU= 0
SPIN	: RL= 0	HEU= 0

McNemar pareado RL vs HEU (limpio): $p=0.0000$
Δ RL-HEU: -70.3pp IC95%: [-71.7, -68.8]

```

Wilcoxon pareado en SUCCESS común (n=14):
pasos          : RL= 999.50 HEU= 646.50 (RL>HEU) p=0.0001 r=1.027
tiempo_s       : RL= 10.00 HEU= 6.46 (RL>HEU) p=0.0001 r=1.027
distancia_final_m : RL= 2.00 HEU= 1.99 (RL>HEU) p=0.7055 r=0.101
energia_total  : RL= 1898.53 HEU= 1795.95 (RL>HEU) p=0.0580 r=0.507

```

```

-----
Análisis pareado limpio: RL vs HEU en BumpyGround con  $\mu=0.3$ , FL=2000 (par alto)
-----

```

```

Excluidos: 17 | Brutos: 100 → Válidos: 83

```

```

RL: 32.5% IC95% [ 23.4, 43.2] (27/83)
HEU: 85.5% IC95% [ 76.4, 91.5] (71/83)

```

```

Distribución sobre conjunto limpio:

```

```

SUCCESS      : RL= 27 HEU= 71
STUCK        : RL= 51 HEU= 6
COLLISION    : RL= 5 HEU= 6
FALL         : RL= 0 HEU= 0
SPIN         : RL= 0 HEU= 0

```

```

McNemar pareado RL vs HEU (limpio): p=0.0000
Δ RL-HEU: -53.0pp IC95%: [-54.2, -51.8]

```

```

Wilcoxon pareado en SUCCESS común (n=26):
pasos          : RL= 920.50 HEU= 680.00 (RL>HEU) p=0.0000 r=0.819
tiempo_s       : RL= 9.21 HEU= 6.80 (RL>HEU) p=0.0000 r=0.822
distancia_final_m : RL= 1.99 HEU= 1.99 (RL>HEU) p=0.0867 r=0.336
energia_total  : RL= 1825.29 HEU= 1906.05 (RL<HEU) p=0.5995 r=0.103

```

7. Tabla resumen y exportación

```

print_header('TABLA RESUMEN — Tasas de éxito y p-valores McNemar')
print(f'\n{"Comparación":50s} {"Δ pp":>8s} {"p McNemar":>12s}')
print('-' * 75)
print('\nA. Intra-sistema (baseline vs perturbado):')
for key, d in results['A_intra'].items():
    print(f' {key:48s} {d["diff_pp"]:>+7.1f} {d["mcnemar_p"]:>11.4f}')
print('\nB. Inter-sistema (RL vs HEU):')
for key, d in results['B_inter'].items():
    print(f' {key:48s} {d["diff_pp"]:>+7.1f} {d["mcnemar_p"]:>11.4f}')
print('\nC. Pareado limpio en BumpyGround:')
for key, d in results['C_paired_clean'].items():
    print(f' {key:48s} {d["diff_pp"]:>+7.1f} {d["mcnemar_p"]:>11.4f}')

def make_serializable(obj):
    if isinstance(obj, dict): return {k: make_serializable(v) for k,v in obj.items()}
    if isinstance(obj, (list,tuple)): return [make_serializable(v) for v in obj]
    if isinstance(obj, (np.integer,)): return int(obj)
    if isinstance(obj, (np.floating,)): return float(obj)
    if isinstance(obj, np.bool_): return bool(obj)
    return obj

with open('resultados_OE9.json', 'w', encoding='utf-8') as f:
    json.dump(make_serializable(results), f, indent=2, ensure_ascii=False)
print('\nResultados guardados en resultados_OE9.json')

```

```

=====
TABLA RESUMEN — Tasas de éxito y p-valores McNemar
=====

```

Comparación	Δ pp	p McNemar

A. Intra-sistema (baseline vs perturbado):		
RL_V3_sticky	+11.0	0.0010
RL_V3_fl2000	+6.0	0.1796
RL_BumpyGround_sticky	-14.0	0.0043
RL_BumpyGround_fl2000	-3.0	0.6291
HEU_V3_sticky	+3.0	0.5078
HEU_V3_fl2000	+0.0	1.0000
HEU_BumpyGround_sticky	+3.0	0.6476
HEU_BumpyGround_fl2000	+4.0	0.3438
B. Inter-sistema (RL vs HEU):		
V3_baseline	-2.0	0.8145
V3_sticky	+6.0	0.0703
V3_fl2000	+4.0	0.3438
BumpyGround_baseline	-39.0	0.0000
BumpyGround_sticky	-56.0	0.0000
BumpyGround_fl2000	-46.0	0.0000
C. Pareado limpio en BumpyGround:		

BumpyGround_sticky_clean	-70.3	0.0000
BumpyGround_fl2000_clean	-53.0	0.0000

Resultados guardados en resultados_OE9.json

No se ha podido establecer conexión con el servicio reCAPTCHA. Comprueba tu conexión a Internet y vuelve a cargar la página para ver otro reCAPTCHA.